



DATABASE SYSTEMS

(CS-151)

Topic: Database Environment

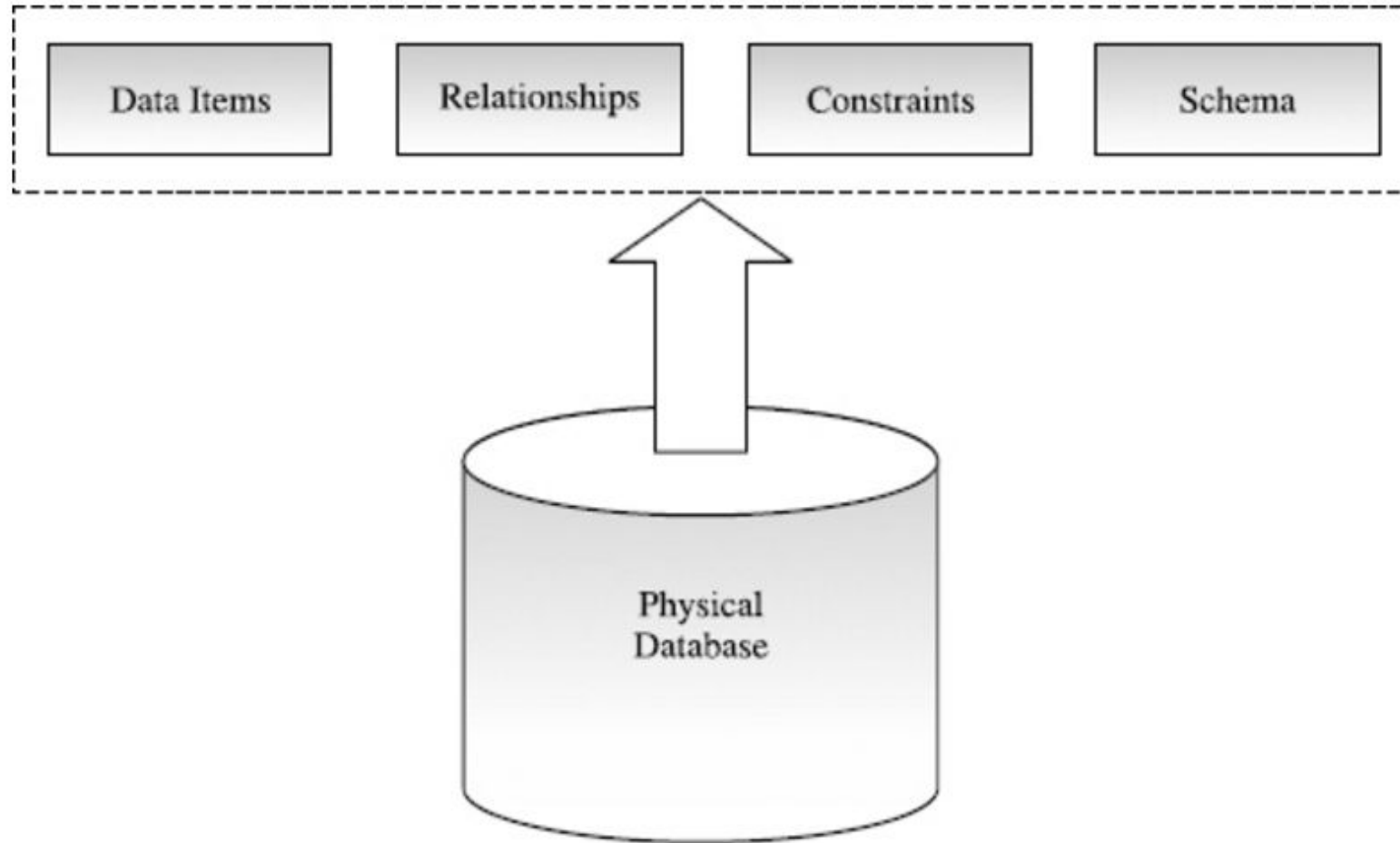
Course Instructor: Munazza



Lecture Overview

- Database Architecture
- Data Independence
- Different aspects of DBMS

Components of Database



Data Abstraction

Database Systems provide users with an abstract view of data, hiding certain details of how data are stored and maintained

Data Abstraction

- Physical Level

- *Describes how data is actually stored*

- Logical Level

- *Describes what data are stored in the database and what relationships exist among those data*

- View Level

- *Describes only part of the entire database by hiding details of data types.*
 - *Views can also hide information (e.g., salary) for security purposes*



DATABASE SYSTEM ARCHITECTURE



Schema

- Plan of the database is known as schema
- Logical representation of DB
- Gives the name of the entities and attributes and also specify the relationship among them
- It includes the definition of database name, the record type and the components that make up those records
- Values fitted into the format changes from instance to instance
 - RDBMS (Tables, Relations)
 - ER (Entities, Relationship)
 - *Entity representation*

6 Database Schema Designs

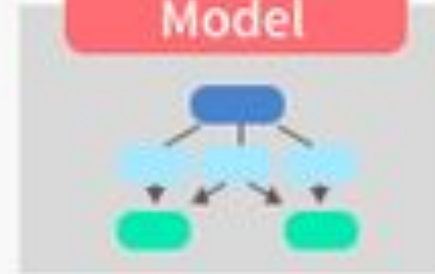
Flat File Model



Hierarchical Model



Network Model



Relational Model



Star Schema



Snowflake Schema



Schema

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

Schem
datab.

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

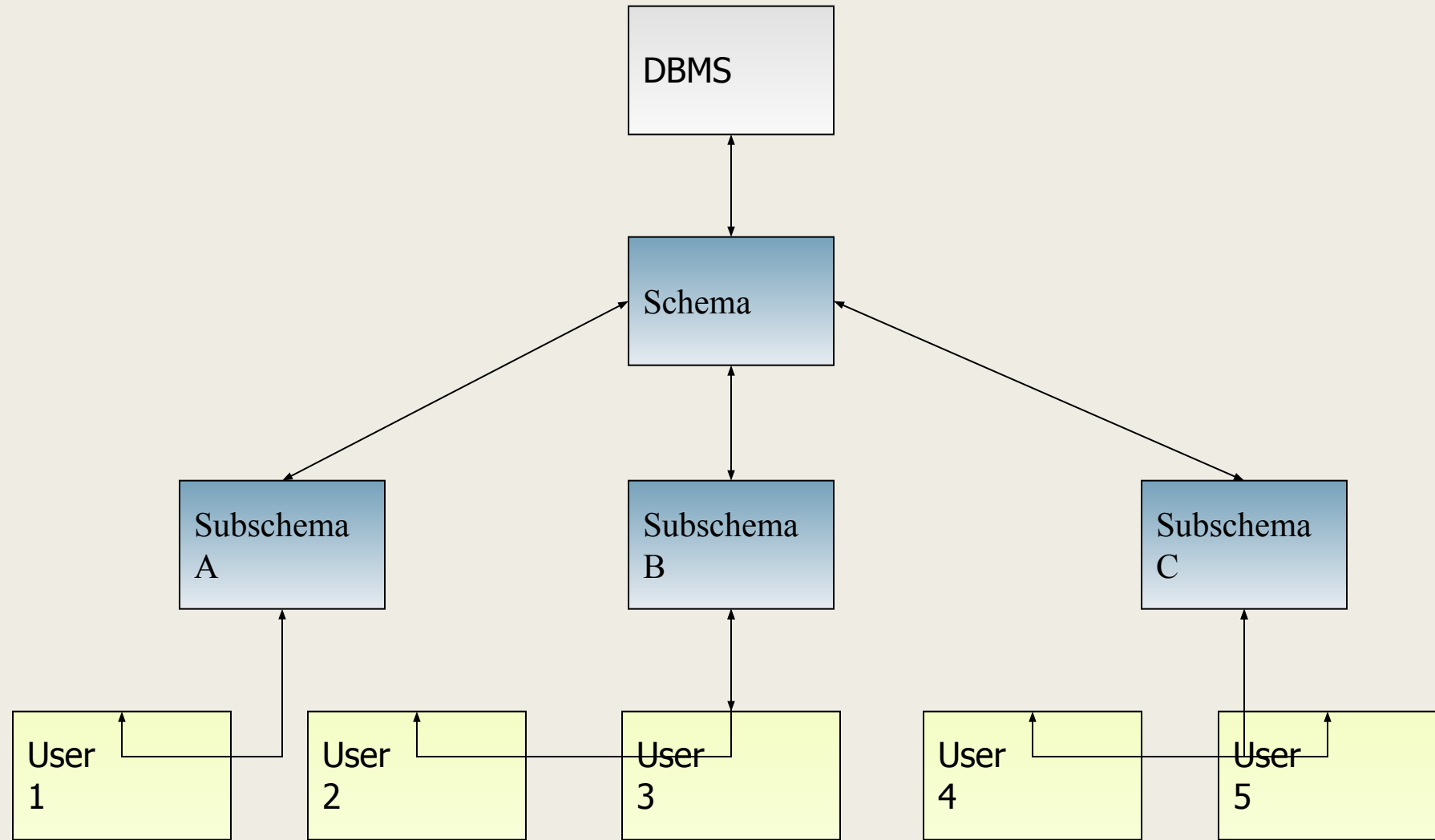
GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

Sub Schema

- Subset of schema
- Inherits the same property as schema has
- Plan for a view is called sub schema
- Refers to the application programmer view of the data item types and record types, which he or she uses
- Gives the users a window through which he or she can view only that part of the database, which is of interest to him
- Different application programmer can have different view of data
- Database administrator ensures that the subschema requested by application programs is derivable from schema

Subschema



Feature	Schema	Subschema
Definition	The overall structure of a database, defining tables, relationships, constraints, and integrity rules.	A subset of the schema that defines the view of the database for a particular user or application.
Scope	Covers the entire database .	Covers only specific parts of the database.
Usage	Used by database administrators (DBAs) to define the entire structure.	Used to provide limited access to certain users or applications.
Access Control	Contains all database objects, including those not accessible by all users.	Restricts users to a particular part of the database.
Example	A university database schema defining tables: <code>Students</code> , <code>Courses</code> , <code>Enrollments</code> , <code>Professors</code> .	A student's subschema allowing access only to <code>Students</code> and <code>Enrollments</code> , but not <code>Professors</code> or <code>Courses</code> details.

Instance

- When the schema framework is filled in the data item values or the contents of the database at any point of time is referred to as instance of the database
- At any point of time, the current state of the database is called instance
- The instance of the database is the values of these variables at any given time.
- The database schema is the design that defines the variables in tables that belong to a particular database.

Instance

STUDENT

Name	Student number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

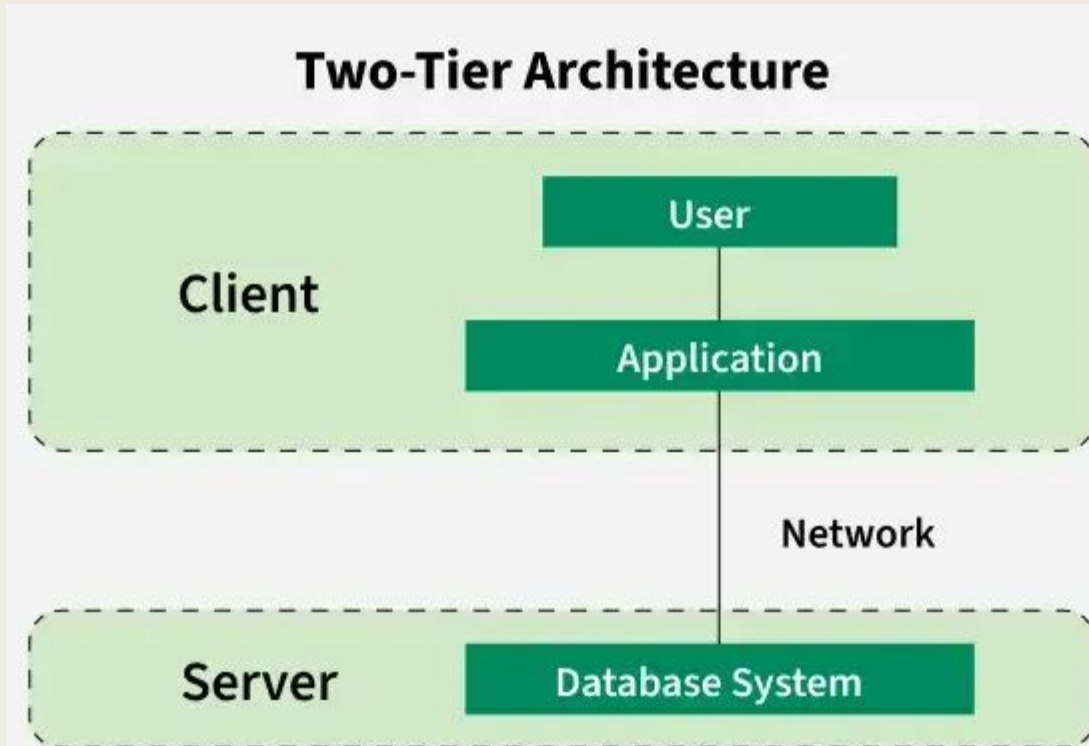
Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Database Architecture

- Database application usually partitioned into:
 - *Two Tier Architecture*
 - *Three Tier Architecture*

Two Tier Architecture

- Application is partitioned into component that resides at the client machine
- The 2-tier architecture is similar to a basic client-server model . The application at the client end directly communicates with the database on the server side.
- Evokes database functionality at server machine through query language statements
- Application program interface standards are used for interaction between client and the server



Two Tier Architecture

For Example: A Library Management System used in schools or small organizations is a classic example of two-tier architecture.

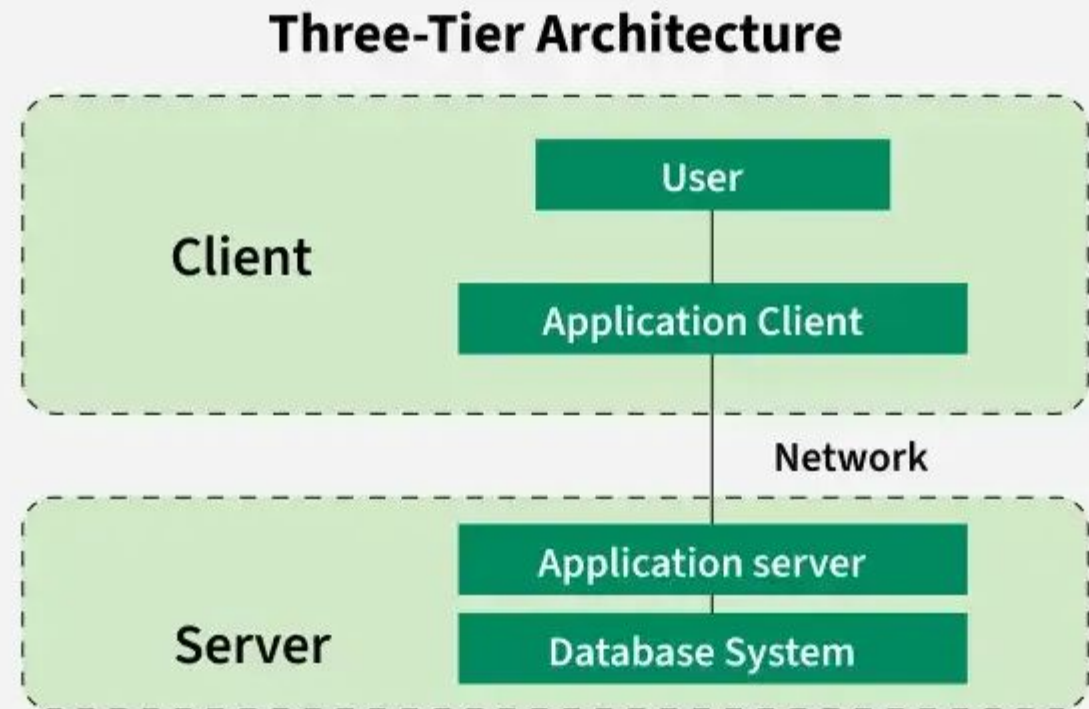
- **Client Layer (Tier 1):** This is the user interface that library staff or users interact with. For example they might use a desktop application to search for books, issue them, or check due dates.
- **Database Layer (Tier 2):** The database server stores all the library records such as book details, user information, and transaction logs.
- The client layer sends a request (like searching for a book) to the database layer which processes it and sends back the result. This separation allows the client to focus on the user interface, while the server handles data storage and retrieval.

Advantages of 2-Tier Architecture

- **Easy to Access:** 2-Tier Architecture makes easy access to the database, which makes fast retrieval.
- **Scalable:** We can scale the database easily, by adding clients or upgrading hardware.

Three Tier Architecture

- There is another layer between the client and the server.
- The client does not directly communicate with the server. Instead, it interacts with an application server which further communicates with the database system and then the query processing and transaction management takes place.
- This intermediate layer acts as a medium for the exchange of partially processed data between the server and the client.
- This type of architecture is used in the case of large web applications.



Three Tier Architecture

For Example: E-commerce Store

User: You visit an online store, search for a product and add it to your cart.

Processing: The system checks if the product is in stock, calculates the total price and applies any discounts.

Database: The product details, your cart and order history are stored in the database for future reference.

Advantages of 3-Tier Architecture

- **Enhanced scalability:** Scalability is enhanced due to the distributed deployment of application servers. Now, individual connections need not be made between the client and server.
- **Data Integrity:** 3-Tier Architecture maintains Data Integrity. Since there is a middle layer between the client and the server, data corruption can be avoided/removed.
- **Security:** 3-Tier Architecture Improves Security. This type of model prevents direct interaction of the client with the server thereby reducing access to unauthorized data.